# Forensic Timeline Consistency and Evidence Constraint Analyzer

## 1. Project Overview

This project is a Python web application intended to help forensic investigators organize evidence events, define relationships and constraints, check timeline consistency, review potential conflicts, and examine alternative timelines when valid solutions exist. The planned stack is Python, Flask, HTML, CSS, Jinja templates, and SQLite. NetworkX and a constraint-solving library will support analysis. The interface will use standard HTML forms and server-rendered pages without custom JavaScript.

## 2. Source Tree and Major Components

The following table explains the proposed source tree. A listed file should only be described as implemented after it has been created and tested in the repository.

| Path | Purpose |
|---|---|
| app.py | Flask application entry point; configures the app and registers routes. |
| config.py | Central configuration such as database path and application settings. |
| requirements.txt | Python dependencies required to install the application. |
| README.md | Project purpose, setup instructions, run commands, and development notes. |
| instance/forensic_timeline.db | Local SQLite database created or initialized by the application. |
| src/__init__.py | Marks src as a Python package. |
| src/database/db.py | SQLite connection and database helper functions. |
| src/database/schema.sql | Table definitions, keys, and database constraints. |
| src/models/evidence_event.py | Evidence-event data fields and basic validation. |
| src/routes/dashboard.py | Dashboard page and summary route. |
| src/routes/events.py | Create, list, edit, and delete evidence events. |
| src/routes/constraints.py | View and define relationships and constraints. |
| src/routes/analysis.py | Start analysis and display results, conflicts, and alternatives. |
| src/routes/datasets.py | Dataset import and export routes. |
| src/services/event_service.py | Event business logic and database coordination. |
| src/services/graph_service.py | Builds a NetworkX graph of evidence relationships. |
| src/services/constraint_service.py | Represents and evaluates supported constraints. |
| src/services/conflict_service.py | Collects violated rules and affected event IDs. |
| src/services/timeline_service.py | Organizes timelines and coordinates alternative generation. |
| src/services/report_service.py | Prepares summaries and report data. |

| | |
|---|---|
| src/algorithms/temporal_checks.py | Checks timestamps, time ranges, durations, and event ordering. |
| src/algorithms/location_checks.py | Checks supported location-related rules. |
| src/algorithms/conflict_detection.py | Identifies incompatible constraints and related events. |
| src/algorithms/alternative_timelines.py | Searches for candidate timelines satisfying supported constraints. |
| src/data_processing/csv_importer.py | Reads and validates CSV event records. |
| src/data_processing/json_importer.py | Reads and validates JSON event records. |
| src/data_processing/nibrs_transformer.py | Maps selected public NIBRS fields to the application's event format. |
| src/ml/event_classifier.py | Optional event-description classifier; not required for core timeline analysis. |
| templates/base.html | Shared HTML layout and navigation. |
| templates/dashboard.html | Dashboard summary and links to major features. |
| templates/events/list.html | Evidence-event listing page. |
| templates/events/add.html | HTML form for adding an event. |
| templates/events/edit.html | HTML form for editing an event. |
| templates/constraints/list.html | Constraint listing and management page. |
| templates/analysis/results.html | Overall analysis status and summary. |
| templates/analysis/conflicts.html | Violated constraints and affected event identifiers. |
| templates/analysis/alternatives.html | Candidate alternative timelines and validation status. |
| templates/reports/detail.html | Detailed analysis report. |
| static/css/style.css | Visual styling for pages, forms, tables, and status messages. |
| datasets/synthetic/ | Controlled sample timelines, including known conflict cases. |
| datasets/samples/ | Small non-sensitive files for import testing. |
| tests/test_events.py | Tests event validation and event operations. |
| tests/test_constraints.py | Tests supported constraint rules. |
| tests/test_conflicts.py | Tests conflict detection against known examples. |
| tests/test_alternative_timelines.py | Tests that generated alternatives satisfy supported constraints. |

## 3. Dependencies and Configuration

Initial dependencies to list in requirements.txt:

Flask
pandas
numpy
networkx
python-constraint2
pytest

SQLite is included with Python. The optional machine-learning extension may require scikit-learn later, but it should not be required for the core application.

### 4. Install and Run in VS Code

Run these commands in the Windows PowerShell terminal from the project root:

```
py -m venv .venv

.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install -r requirements.txt

python app.py
```

Open the local address printed by Flask in the terminal, commonly http://127.0.0.1:5000/. These instructions assume app.py and requirements.txt have been implemented. If the application uses a different startup command, document that command in README.md.

### 5. Database Setup

The database helper should connect to instance/forensic_timeline.db and initialize the database from src/database/schema.sql. The planned schema will cover evidence events, event relationships, constraints, analysis runs, conflicts, and alternative timelines. Enable SQLite foreign-key checks and validate them during testing. Do not store private forensic records in the public repository.

### 6. Execution Status and Verification

The document describes the planned implementation foundation; it does not prove that every listed module already exists or works. Before submission, run the application and record the actual results.

- Install dependencies with pip install -r requirements.txt.
- Start the application and confirm Flask reports that it is running.
- Open the dashboard in a browser.
- Create a test event, restart the app, and confirm the record persists in SQLite.
- Submit an invalid time range and confirm it is rejected clearly.
- Run a known synthetic conflict case and check the reported event IDs and rule.
- Run pytest and record the actual pass/fail counts.

### 7. Architecture and Technology Decisions

| Decision | Reason |
|---|---|
| Flask instead of Streamlit | The original proposal named Streamlit. Flask is chosen for this implementation plan because it supports separate HTML templates, CSS, and conventional form-based pages. |

| | |
|---|---|
| SQLite | Suitable for a local laptop application without a separate database server. A server database can be considered if multi-user deployment is later required. |
| Constraint solving as the core | Explicit constraints and graph algorithms directly address timeline consistency; machine learning is optional and must not decide whether a timeline is consistent. |
| No custom JavaScript | HTML forms submit to Flask and the server renders updated pages. Use static or server-generated visualizations. |
| Modular source tree | Separating routes, services, algorithms, database access, and data processing makes the code easier to test and maintain. |

## 8. Week 2 Implementation Plan

| **Work item** | **Planned result** |
|---|---|
| Flask foundation | Verify the entry point, configuration, route registration, base template, dashboard, and CSS. |
| SQLite integration | Implement database connection helpers, schema initialization, and foreign-key enforcement. |
| Evidence-event CRUD | Implement create, list, edit, and delete operations with input validation. |
| Basic constraints | Support initial rules such as one event occurring before another. |
| Conflict reporting | Run synthetic examples and report affected event IDs and violated rules. |
| Automated tests | Test valid events, invalid time ranges, constraints, and known conflicts. |
| Execution evidence | Capture real terminal output, browser screenshots, and test results. |